Robot State Estimation

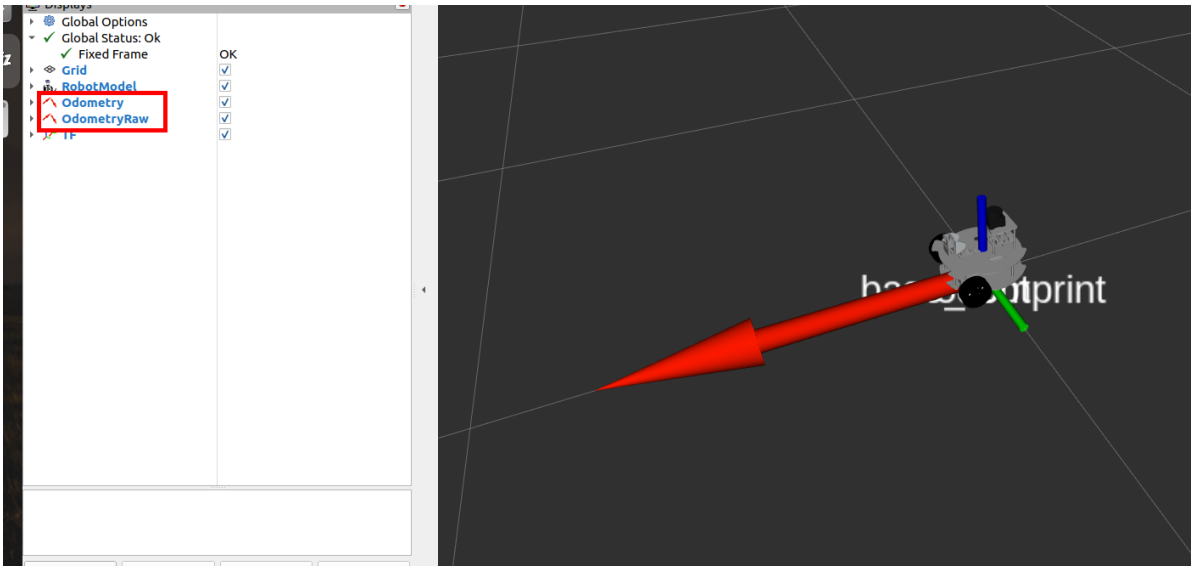
1. Course Content

- Fuse multi-sensor data from the IMU and wheel encoders to obtain the fused odometry data odom for estimating the robot's moving position
- Use rviz to observe the trajectory difference between the wheel odometry (blue arrows) and the odometry obtained after fusing IMU data (red arrows)

2. Preparation

- Start the state estimation node

```
ros2 launch microros_v2_bringup state_estimation.launch.py
```



- Start the keyboard control node

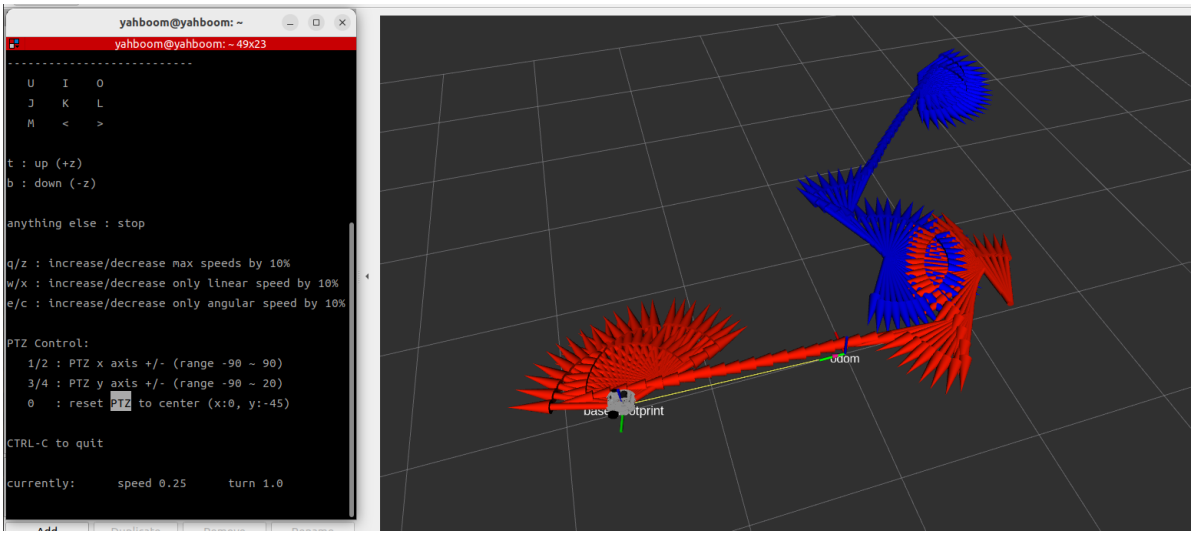
```
ros2 run common_demo keyboard_control
```

3. Demonstration

Note: If the odometry data is incorrect, you can press the reset button to restart the car

- Use the keyboard to move the car. You can observe that the odometry trajectories of odom (red arrows) and odom_raw (blue arrows) gradually diverge. EKF combines the wheel

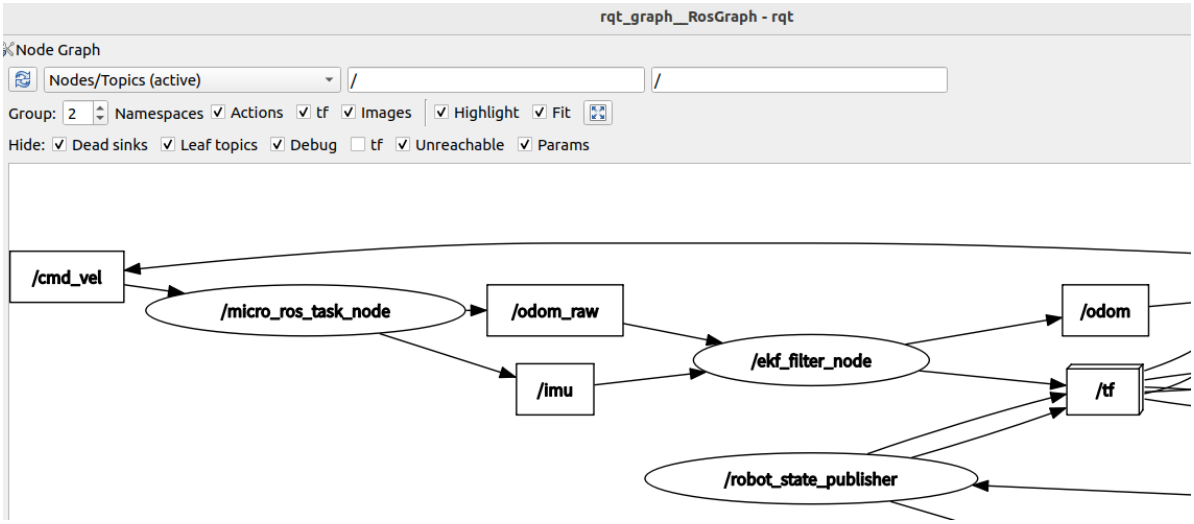
odometry and IMU data to obtain more stable odometry data



3. View the Node Graph

```
rqt_graph
```

- If the node graph is not displayed, click the refresh button in the top-left corner
- You can see that /odom_raw is the wheel odometry computed from the chassis motor encoder pulses, /imu is the IMU data, and the ekf_filter_node node subscribes to both and publishes the fused odometry /odom



4. Source Code Analysis

- car_base launch file

```
ros2 launch microros_v2_bringup car_base.launch.py
```

`car_base.launch.py` is the basic launch file of the robot chassis. It starts the following nodes in order:

1. `camera_driver`: ESP32 camera driver. It builds the `http://<ip>:81/stream` video stream address based on the `camera_ip` parameter, and only starts when `use_camera:=true` and the model is not `basic_car`.
2. `micro_ros_agent`: micro-ROS agent. It communicates with the control board over UDP port 8090, bridging the MCU and ROS 2 topics.

3. `base_driver`: Chassis driver node. It loads the `common.yaml` common parameters and configures the wheel track, wheel diameter, etc. according to the model.
4. `robot_state_publisher`: Parses `microros_v2.urdf.xacro` through xacro and publishes the robot TF transforms.
5. `joint_state_publisher`: Subscribes to `ptz_status` and publishes the gimbal joint states, so the gimbal joints in the URDF can follow the real angles.
6. `ekf_filter_node`: The EKF node of the `robot_localization` package. It loads `ekf.yaml` to fuse `/odom_raw` and `/imu`, and outputs the fused `/odom`.

```
def generate_launch_description():
    #bring camera and select robot  model
    camera_type=GetRobotType()
    # ptz | no_ptz | basic_car

 robot_urdf=os.path.join(get_package_share_directory('robot_description'),'urdf'
,'microros_v2.urdf.xacro')
    robot_description = Command(
        ["xacro", " ", robot_urdf," ", "camera_type:=", camera_type] )
    # Declare arguments
    declared_arguments = []
    # Sensor control arguments
    declared_arguments.append(
        DeclareLaunchArgument(
            'use_camera',
            default_value='true',
            description='Whether to launch camera'
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            'camera_ip',
            default_value='192.168.12.32',
            description='IP address of the ESP32 camera'
        )
    )

    declared_arguments.append(
        DeclareLaunchArgument(
            'micro_ros_agent_verbose',
            default_value='3',
            description='micro-ROS agent verbosity level (0-6)'
        )
    )

    # camera_url format: http://<camera_ip>:81/stream
    camera_url = PythonExpression(["'http://' + '",
LaunchConfiguration('camera_ip'), "' + ':81/stream'"])
    common_params =
os.path.join(get_package_share_directory('microros_v2_bringup'),'config','common
.yaml')

    camrea_drive=Node(
        package='microros_v2_bringup',
        executable='camera_driver',
        output='screen',
        parameters=[{'camera_url': ParameterValue(camera_url, value_type=str),
```

```
 'img_save_path':os.path.join(os.path.expanduser('~'),'microros_ws','cache','ima
ge.png'),
        'ost_file':
os.path.join(get_package_share_directory('microros_v2_bringup'),'config','ost.ya
ml')}],
        condition=IfCondition(PythonExpression(["'",
LaunchConfiguration('use_camera'), "' == 'true' and '", camera_type, "' !=
'basic_car'"]))
        )

    mircoros_agent=Node(
        package='micro_ros_agent',
        executable='micro_ros_agent',
        output='screen',
        arguments=['udp4', '--port', '8090', '-v',
LaunchConfiguration('micro_ros_agent_verbose')],
    )

    base_driver=Node(
        package='microros_v2_bringup',
        executable='base_driver',
        name='base_driver',
        parameters=[common_params,
        {'camera_type': camera_type}],
        output='screen',
    )

    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{'robot_description': ParameterValue(robot_description,
value_type=str)}],
        arguments=['--ros-args','--log-level','warn']
    )
    joint_publisher_node=Node(
        package='joint_state_publisher',
        executable='joint_state_publisher',
        parameters=[{'source_list': ["ptz_status"]}],
        arguments=['--ros-args','--log-level','warn']
    )
    #ekf_odom
    ekf_odom=Node(
        package='robot_localization',
        executable='ekf_node',
        name="ekf_filter_node",
        parameters=
[os.path.join(get_package_share_directory('microros_v2_bringup'),'config','ekf.y
aml')],
        remappings=[('/odometry/filtered', '/odom')]
    )

    return LaunchDescription([
        *declared_arguments,
        mircoros_agent,
        camrea_drive,
        robot_state_publisher_node,
        joint_publisher_node,
```

```
        base_driver,
        ekf_odom,
    ])
```

- state_estimation launch file

`state_estimation.launch.py` adds rviz visualization on top of `car_base.launch.py` to compare the trajectories of the raw wheel odometry and the fused odometry:

1. `IncludeLaunchDescription`: Reuses `car_base.launch.py`, passing `use_camera:=false` to disable the camera to save resources, and `log_level:=info` to enable detailed logging.
2. `rviz2`: Loads the `state_estimation.rviz` configuration file, simultaneously displaying the two odometry trajectories `odom` (red) and `odom_raw` (blue), making it easy to intuitively compare the EKF fusion effect.

```
    rviz_config =
os.path.join(get_package_share_directory('microros_v2_bringup'),'config','state_
estimation.rviz')
    carbase=IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(get_package_share_directory('microro
s_v2_bringup'),'launch','car_base.launch.py')),
            launch_arguments={'log_level': 'info',
                              'use_camera': 'false'
                              }.items())

    rviz_node = Node(
        package = 'rviz2',
        executable = 'rviz2',
        name='rviz2',
        arguments=['-d', rviz_config,
                   '--ros-args', '--log-level', 'info'],
        output='screen'
    )
```

- ekf parameter file

```
$HOME/microros_ws/src/microros_v2_bringup/config/ekf.yaml
```

`ekf.yaml` is the core configuration of the EKF fusion node. The key parameters are explained as follows:

- **Global parameters**: `frequency: 20.0` means the EKF runs at 20Hz; `two_d_mode: true` limits it to planar motion (ignoring z/roll/pitch); `publish_tf: true` automatically publishes the `odom → base_footprint` transform.
- `odom0: /odom_raw`: Wheel odometry input. The 15-element boolean array `odom0_config` corresponds to `[x,y,z, roll,pitch,yaw, vx,vy,vz, vroll,vpitch,vyaw, ax,ay,az]` and indicates whether each quantity participates in the fusion. Here only `vx` (forward velocity) and `vyaw` (yaw angular velocity) are enabled, because a differential-drive chassis can only reliably observe these two quantities.

- `imu0: /imu`: IMU input. `imu0_config` only enables `yaw` (yaw angle), using the IMU's magnetometer/gyroscope to correct the heading drift of the wheel odometry; `imu0_relative: true` uses the first IMU data as the reference baseline, avoiding jumps caused by the absolute heading being inconsistent with the odometry's initial orientation.
- **Fusion strategy**: The odometry provides translational velocity and the IMU provides heading angle; the two complement each other — the odometry is accurate in the short term but accumulates heading drift, while the IMU heading is stable in the long term but noisy in the short term. EKF combines the advantages of both to obtain a more stable `/odom`.

```
ekf_filter_node:
    ros__parameters:
        use_sim_time: false
        frequency: 20.0
        sensor_timeout: 0.3
        two_d_mode: true
        transform_time_offset: 0.0
        transform_timeout: 0.05
        print_diagnostics: false
        debug: false
        publish_tf: true
        publish_acceleration: false
        reset_on_time_jump: true
        odom_frame: odom                # Defaults to "odom" if unspecified
        base_link_frame: base_footprint  # Defaults to "base_link" if
unspecified
        world_frame: odom               # Defaults to the value of odom_frame
if unspecified

        odom0: /odom_raw
        odom0_config: [false, false, false,
                       false, false, false,
                       true, false, false,
                       false, false, true,
                       false, false, false]

        odom0_queue_size: 3
        odom0_nodelay: true
        odom0_differential: false
        odom0_relative: false
        odom0_pose_rejection_threshold: 20.0
        odom0_twist_rejection_threshold: 5.0

        imu0: /imu
        imu0_config: [false, false, false,
                      false, false, true,
                      false, false, false,
                      false, false, false,
                      false, false, false]

        #        [x_pos   , y_pos    , z_pos,
        #         roll    , pitch    , yaw,
        #         x_vel   , y_vel    , z_vel,
        #         roll_vel, pitch_vel, yaw_vel,
        #         x_accel , y_accel  , z_accel]
```

```
        imu0_nodelay: true
        imu0_differential: false
        imu0_relative: true
        imu0_queue_size: 5
        imu0_pose_rejection_threshold: 5.0                # Note the difference
in parameter names     # 0.8
        imu0_twist_rejection_threshold: 5.0               # 0.8
        imu0_linear_acceleration_rejection_threshold: 10.0  #
        imu0_remove_gravitational_acceleration: true
```